n=3

arr[0]	AAAB 10
[1]	AABA 10
[2]	ABAA 10
[3]	

0-1 | 0-1
1-2
0

char temp[20];

for(i=n-1; i>0; i--)
{
 for(j=0; j<i; j++)
  if(strcmp(arr[j], arr[j+1])<0)
  { strcpy(temp, arr[j]);
   strcpy(arr[j], arr[j+1])
   strcpy(arr[j+1], temp);
  }
}

3
0-1
1-2
2-3

BADC
BDAC
BDCA

2
0-1
1-2

DDCA
DBCA
DCBA

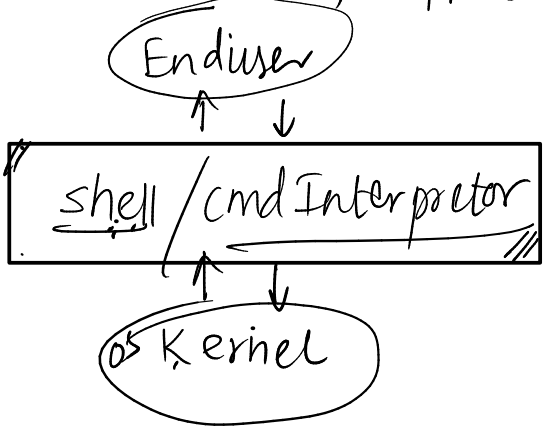
1
0-1

Command Line Argument.

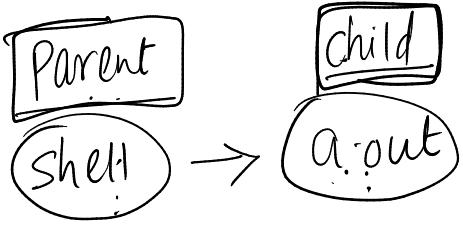
-> Data supply via cmd line, at the time of app getting Loaded into memory

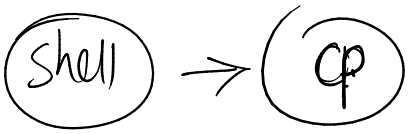
[-----] $ ls ↵

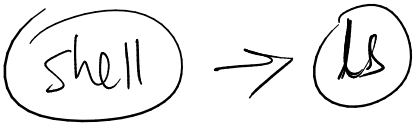
[ ] $ ./a.out ↵

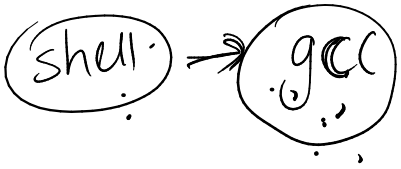












Data Input to the program

- compile time
- Run time
- cmd line/load time

---

void / int
main( )
{
=
}

- main(int argc) {
=
}

- main(int x, char **y, char **z)
{ argc, argv, envp
=
z
}

Environmental pointer

- main (int argc, char **argv)
{
=
}

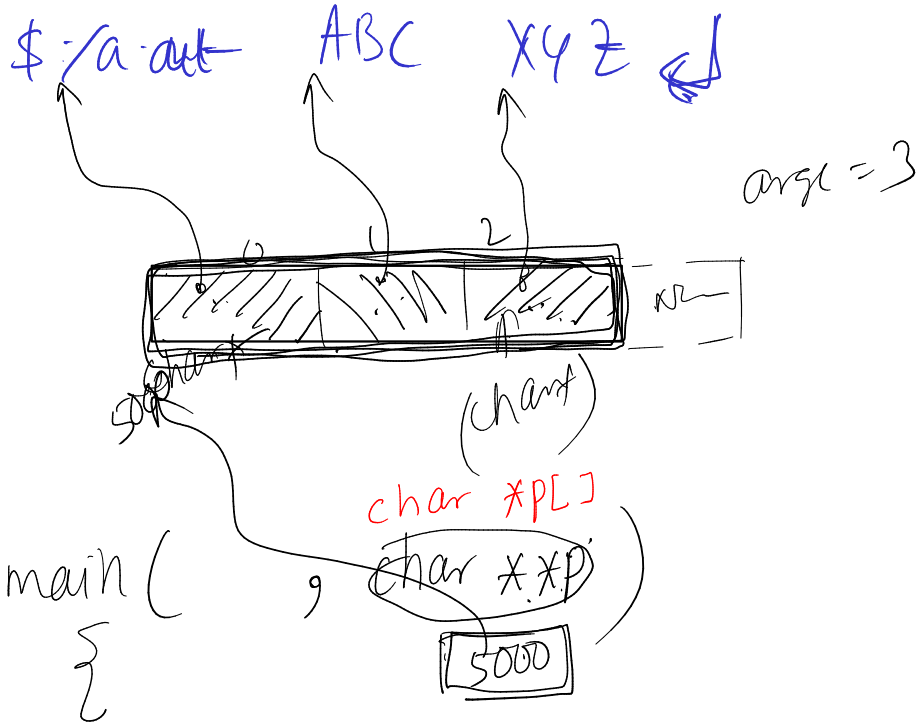
$ ./a.out ↵

main(int argc)
=1

$ ./a.out 123 abc xyz

main(int argc
=4

$ ./a.out ABC XYZ ↵



$ cp ↵

$

$ gcc ↵

$

---

main() {
int arr[5];
=
f1(arr);
}

int *pb[]
f1(int **pb)
{
=
}

Create a Cmd-line Calculator like expr

$ ./a.out 11+22-33 ↵

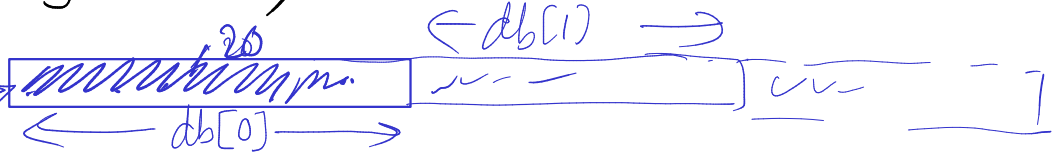
$ ./calci 11 + 22 ↵

33

./calci 11 + ab ↵

Name Database with Dyn. Mem Allocation

char (*db)[20] = NULL;



cnt = 0

db



1

2

on Demand Allocation

choice : i> .{

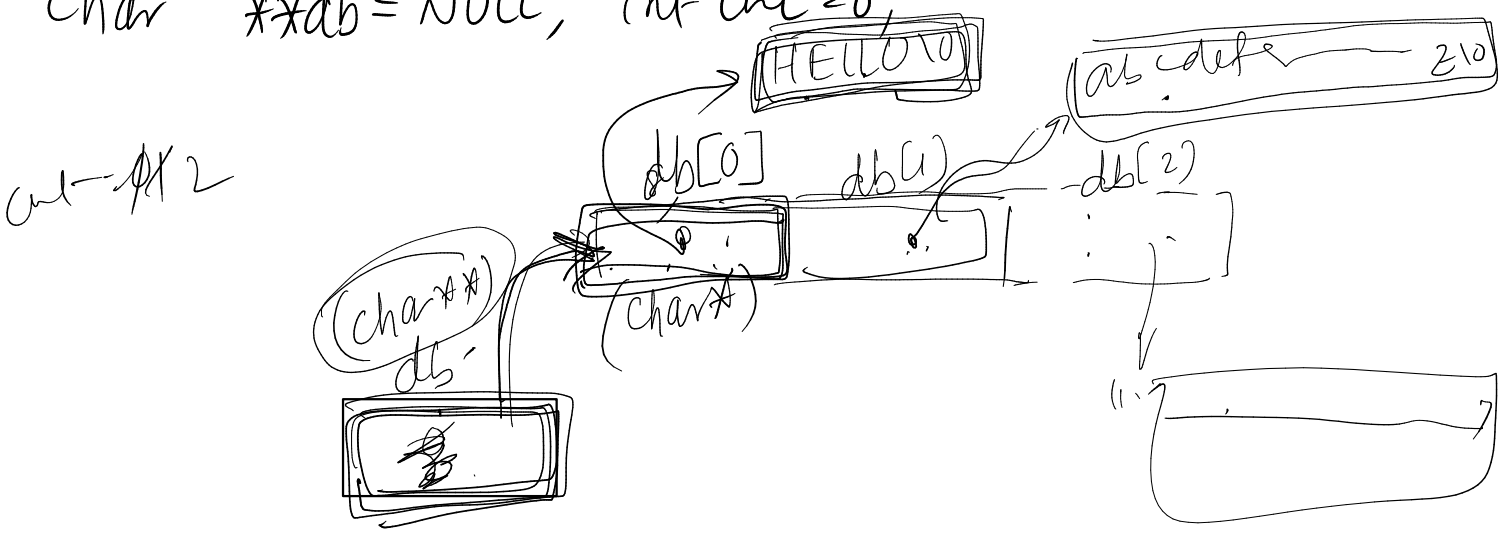
db = realloc (db, (cnt+1) * sizeof (*db));

gets (db[cnt]);

cnt++;

}

char **db = NULL; int cnt=0;



cnt=2

input( ){

db = realloc(db, (cnt+1) * sizeof(char*));

db[cnt] = getString( );

cnt++;

}